

OpenAlgo Remote Access via Tailscale (Office PC ↔ Home PC ↔ Mobile)

Objective

Access the OpenAlgo frontend running on the **Office PC** from:

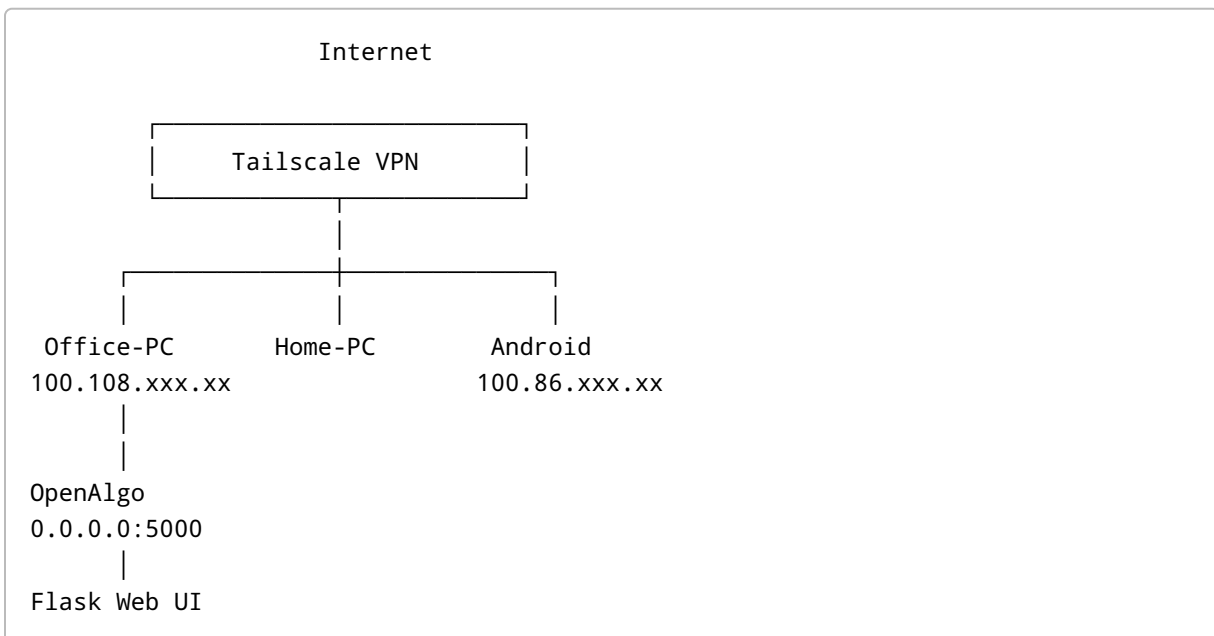
- Home-PC
- Android Mobile

without

- Port forwarding
- Static IP
- Domain name
- Cloudflare Tunnel

using **Tailscale**.

Final Architecture



All devices join the same private Tailscale network.

No public IP is exposed.

Requirements

Office-PC

- Windows
- OpenAlgo installed
- Tailscale installed
- Logged into same Tailscale account

Home-PC

- Windows
- Tailscale installed
- Logged into same account

Mobile

- Android
- Tailscale installed
- Logged into same account

Step 1 — Install Tailscale

Install Tailscale on

- Office-PC
- Home-PC
- Mobile

Login using the same account.

Example

shreyas.stock.91@gmail.com

(or whichever account is used)

Step 2 — Verify Tailscale

Office-PC

```
tailscale status
```

Example

```
100.108.179.50    Office-PC
100.86.146.33     Android
100.xxx.xxx.xxx   Home-PC
```

Write down

```
Office-PC Tailscale IP
```

Example

```
100.108.179.50
```

This IP normally remains stable while the device stays in your tailnet.

Step 3 — Modify OpenAlgo

Edit

```
.env
```

Change

```
FLASK_HOST_IP='127.0.0.1'
```

to

```
FLASK_HOST_IP='0.0.0.0'
```

Nothing else is required.

Step 4 — Restart OpenAlgo

```
uv run app.py
```

Expected banner

```
Web App
```

```
http://10.xx.xx.xx:5000
```

Do NOT worry if it shows the office LAN IP.

The important part is that the server is no longer bound to localhost.

Step 5 — Verify Server Binding

Run

```
netstat -ano | findstr :5000
```

Expected

```
TCP 0.0.0.0:5000 LISTENING
```

NOT

```
127.0.0.1:5000
```

If you see

```
0.0.0.0
```

the server is reachable from other devices.

Step 6 — Windows Firewall

Run PowerShell as Administrator

```
New-NetFirewallRule `
  -DisplayName "OpenAlgo 5000" `
  -Direction Inbound `
  -Protocol TCP `
  -LocalPort 5000 `
  -Action Allow
```

Verify

```
Get-NetFirewallRule "OpenAlgo 5000"
```

Step 7 — Local Verification

On Office-PC browser

```
http://127.0.0.1:5000
```

Should work.

Then

```
http://<Office-LAN-IP>:5000
```

Example

```
http://10.15.45.12:5000
```

Should also work.

Step 8 — Remote Verification

From Home-PC

```
http://100.108.179.50:5000
```

Should open.

From Mobile

```
http://100.108.179.50:5000
```

Should also open.

Verify Tailscale

Office-PC

```
tailscale status
```

Expected

```
Office-PC  
Home-PC  
Mobile
```

All should be visible.

Connection Checklist

Office-PC

✓ OpenAlgo running

```
uv run app.py
```

✓ Tailscale connected

```
tailscale status
```

✓ Firewall rule present

✓

```
netstat -ano | findstr :5000
```

shows

```
0.0.0.0:5000
```

Home-PC

Tailscale Connected

```
tailscale status
```

Browser

```
http://100.108.179.50:5000
```

Mobile

Open Tailscale

Status

```
Connected
```

Browser

```
http://100.108.179.50:5000
```

Office-PC Test Script

Create

```
testscript.bat
```

```
@echo off

echo =====
echo OpenAlgo Connectivity Test
echo =====

echo.
echo ---- Tailscale ----
tailscale status

echo.
echo ---- Listening Port ----
netstat -ano | findstr :5000

echo.
echo ---- Ping Mobile ----
ping 100.86.146.33

pause
```

Home-PC Test Script

```
@echo off

echo =====
echo Remote Connectivity Test
echo =====

echo.

echo ---- Ping Office ----
ping 100.108.179.50

echo.
echo If ping succeeds,
```



```
echo Open

echo.
echo http://100.108.179.50:5000

pause
```

Python Test Script

```
import requests

url = "http://100.108.179.50:5000"

try:
    r = requests.get(url, timeout=5)
    print("SUCCESS")
    print("Status:", r.status_code)
except Exception as e:
    print("FAILED")
    print(e)
```

Run

```
python testscript.py
```

Expected

```
SUCCESS
Status: 200
```

Troubleshooting

Browser cannot connect

Run

```
netstat -ano | findstr :5000
```

Must show

```
0.0.0.0:5000
```

Ping works

Browser doesn't

Usually Windows Firewall.

Re-create firewall rule.

Browser opens on Office-PC only

Usually

```
FLASK_HOST_IP='127.0.0.1'
```

Change to

```
0.0.0.0
```

Restart OpenAlgo.

Tailscale devices not visible

Run

```
tailscale status
```

Login again if required.

Security Notes

Using Tailscale means

- No public IP exposure
 - No router configuration
 - No port forwarding
 - End-to-end encrypted communication
 - Only authenticated devices in your tailnet can reach the server
-

Future Enhancements

1. HOST_SERVER

```
HOST_SERVER='http://100.108.179.50:5000'
```

Purpose

Defines the canonical URL of the OpenAlgo server.

Benefits

- Generates correct absolute URLs.
 - Useful for APIs that return links.
 - Helps integrations reference the server consistently.
 - Useful for future remote services (webhooks, MCP, reverse proxies, etc.).
-

2. CORS_ALLOWED_ORIGINS

```
CORS_ALLOWED_ORIGINS='http://100.108.179.50:5000'
```

Purpose

Allows browser-based JavaScript clients hosted at that origin to call OpenAlgo's APIs.

Benefits

- Enables custom dashboards.
- Allows React/Vue/Angular or plain HTML applications running on another machine to communicate with OpenAlgo.

- Prevents browser CORS errors when consuming the REST API from a different origin.

For the built-in OpenAlgo UI, changing this is generally **not required**.

3. WEBSOCKET_HOST

Current

```
WEBSOCKET_HOST='127.0.0.1'
```

Purpose

Controls where the WebSocket server listens.

Current Behaviour

Only local processes on the Office-PC can connect directly.

Enhancement

Changing to

```
WEBSOCKET_HOST='0.0.0.0'
```

would allow remote clients over Tailscale to establish WebSocket connections directly.

Possible Uses

- Real-time dashboards.
- Live tick streaming.
- External monitoring tools.
- Remote strategy control panels.
- Mobile apps displaying live market data.

This should only be enabled if you specifically need remote WebSocket access.

4. WEBSOCKET_URL

Current

```
WEBSOCKET_URL='ws://127.0.0.1:8765'
```

Purpose

Specifies the WebSocket endpoint advertised to clients.

If WebSocket is exposed through Tailscale, it could become:

```
ws://100.108.179.50:8765
```

or, when secured with a reverse proxy:

```
wss://your-server:8765
```

Benefits

- Remote live streaming.
- Bidirectional communication.
- Lower latency than repeated HTTP polling.
- Supports advanced dashboards and automation clients.

Keep the current value unless you intentionally expose the WebSocket service.